

Python for Data Science — Tutorial 2 / 10

```
python3

>>> ages = ["28", "35", "42"]
>>> result = ages[0] + ages[1] + ages[2]
>>> print(result)
"283542"

# no error. just wrong.
```

YOUR ANALYSIS CAN BE WRONG WITHOUT CRASHING

Types encode intention. Python needs to know what you're holding.

THE SILENT BUG

Python Will Give You a Wrong Answer Without Telling You

IMPORTED CSV

ages.csv	
NAME	AGE
Tarun	"28"
Priya	"35"
Carlos	"42"

Looks like numbers. They're strings.

TERMINAL OUTPUT

```
python3

>>> result = "28" + "35" + "42"
>>> print(result)
"283542"

# No error. No warning.
```

No warning. No crash. **Just a wrong number.**

- Summing a string column doesn't error — it concatenates
- "28" + "35" + "42" = "283542" in Python
- Types encode intention. Python needs to know what you mean.

The 4 Primitive Types

int	Whole numbers	28, 100, -5
float	Decimals	3.14, 0.5
str	Text	"42" ≠ 42
bool	True / False only	

```
types.py

age = 28          # int
salary = 95000.50 # float
name = "Tarun"    # str
is_active = True  # bool

# The silent bug
ages_str = ["28", "35", "42"]
print("28" + "35" + "42") # "283542"

# Correct
ages_int = [28, 35, 42]

print(sum(ages_int)) # 105
```

DIAGNOSE BEFORE YOU CALCULATE

Check and Convert Types

- `type(x)` tells you what Python thinks
- `int()`, `float()`, `str()` convert types
- Always check before calculating

First diagnostic habit.

Run `type()` on every column before you touch the data.

```
type_check.py

x = "42"
print(type(x))      # <class 'str'>

x_int = int(x)
print(type(x_int))  # <class 'int'>
print(x_int + 1)    # 43

# Diagnose before you calculate
ages = ["28", "35", "42"]
print(type(ages[0])) # <class 'str'>

ages_clean = [int(a) for a in ages]
print(sum(ages_clean)) # 105
```

Lists: Ordered Sequences

- Index from `0`, slice with `[start:end]`
- `append()` adds to the end
- Comprehensions: filter in one line

SAME RESULT, TWO WAYS

```
high = []  
for s in scores:  
    if s > 85: high.append(s)
```



```
high = [s for s in scores if s > 85]
```

```
lists.py  
  
scores = [72, 88, 91, 64, 95, 83]  
  
# Indexing  
print(scores[0])      # 72  
print(scores[-1])     # 83  
  
# Slicing (end excluded)  
print(scores[1:4])     # [88, 91, 64]  
  
# List comprehension  
high = [s for s in scores if s > 85]  
  
print(high)           # [88, 91, 95]
```


Dictionaries: Named Records

dicts.py

```
employee = {"name": "Tarun", "age": 28}

# Safe access - no KeyError
dept = employee.get("dept", "Unknown")

# List of dicts - dataset shape
employees = [
    {"name": "Tarun", "salary": 95000},
    {"name": "Priya", "salary": 87000},
    {"name": "Carlos", "salary": 102000},
]

total = sum(e["salary"] for e in employees)
print(total) # 284000
```

- Key-value pairs — one record per dict
- `.get()` for safe access
- List of dicts = natural dataset shape

HOW CSVS MAP TO PYTHON

Row 0	→	<code>{"Tarun", 95k}</code>
Row 1	→	<code>{"Priya", 87k}</code>
Row 2	→	<code>{"Carlos", 102k}</code>

Functions: Write Once, Trust Forever

functions.py

```
def calculate_average(values):
    if not values:
        return 0
    return sum(values) / len(values)

scores = [72, 88, 91, 64, 95]
print(calculate_average(scores))  # 82.0

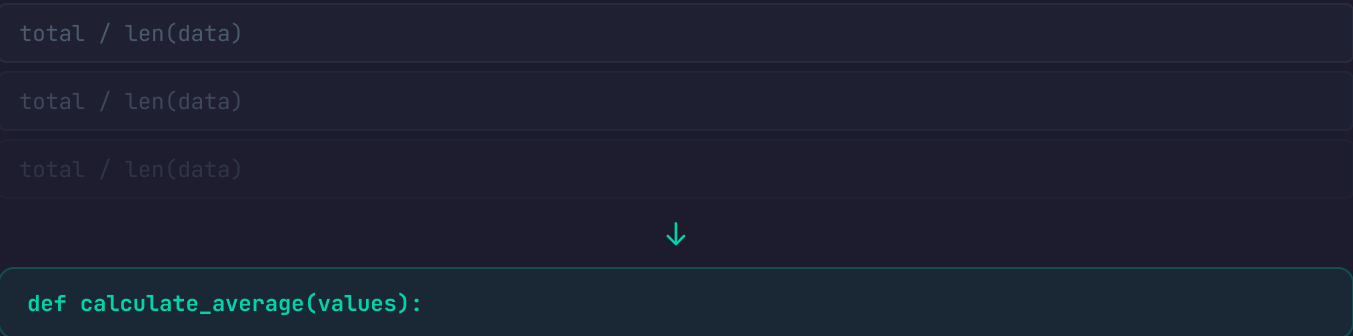
import statistics

def is_outlier(val, data, threshold=2.0):
    mean = calculate_average(data)
    std  = statistics.stdev(data)
    return abs(val - mean) > threshold * std

print(is_outlier(150, scores))  # True
print(is_outlier(88, scores))  # False
```

- `def` — reusable, testable
- Default args: `threshold=2.0`
- One function replaces 3 notebook copies

ONE FUNCTION. THREE FIXED.



WHAT COMES NEXT

What's Next: Tutorial 3 — NumPy



- These structures scale to millions of rows with NumPy
- Arrays replace lists for numerical work
- Everything from Tutorial 2 underpins Tutorial 3

Tutorial 3: NumPy arrays. Same logic, ten thousand times faster.